



Closing the gap between build evidence & compliance enforcement

(An introduction to Conforma)

Cuiping Huo

Senior Software Quality Engineer
Red Hat

Simon Baird

Principal Software Engineer
Red Hat

What our pipelines produce

Modern pipelines create a lot of valuable information, for example:

- **SBOMs.** Lists of dependencies used in the artifact, in SPDX or CycloneDX format.
- **Vulnerability scan reports.** Lists of known CVEs impacting the artifact.
- **SLSA provenance.** Signed attestations showing in detail how, where, and when the artifact was built.
- **Signatures.** Cryptographic signatures verifying the artifact's origin and integrity.

What are we doing with all that data?

Some questions that might be going unanswered:

■ SBOMs

- Is the SBOM complete and accurate?
- Are the packages being sourced from approved package registries and fetched using approved tooling?
- Does the dependency list meet policies around licensing and export restrictions?

What are we doing with all that data?

Some questions that might be going unanswered:

- **Vulnerability scan reports**

- Does the artifact meet your company's policies for being shippable?
 - (Whatever they might be...)
- Was the scanner run recently enough to be considered still current?

What are we doing with all that data?

Some questions that might be going unanswered:

■ SLSA Provenance

- Was this container truly built by a trusted CI builder?
- Was it built using known and trusted methods?
- Were any checks or tests run during the build?
- If so did they report any problems?

What are we doing with all that data?

Some questions that might be going unanswered:

- **Signatures**

- Do the artifact and its attestations pass a signature check?
 - (If we don't check then tampering may go undetected)

Real-world threats

Four compliance gaps show why static green checks are misleading

Stale/incomplete SBOM

Looks like: Valid SPDX attached.

Reality: Generated before final dependencies; 3 components missing.

Source-provenance mismatch

Looks like: Signed provenance.

Reality: Signed, but the source in the provenance doesn't match what was declared

Post-scan vulnerability

Looks like: Clean scan Tuesday.

Reality: Critical CVE published Wednesday; deployment is Thursday.

License not on the allow-list

Looks like: License scan passed.

Reality: A transitive dependency carries a copyleft license your policy never allowed.

"The compliance data was technically there. Nobody checked it against policy at the gate."



The verification gap

Pipeline evidence is useless if nothing acts on it

- Collecting the data is the first step, but we also need to pay attention to what was collected.
- Catch policy violations before publishing/deploying/promoting your artifact.
- Let's check many things and “gate” releases if something isn't right

“If nothing acts on the evidence, pipelines can still ship non-compliant artifacts.”



The background is a solid light purple color. Overlaid on this are several geometric shapes in different shades of purple. A large, dark purple circle is positioned on the left side. A medium-sized, medium-purple circle is on the right side. In the center, there is a square of a slightly darker purple shade. The text 'Introducing Conforma' is centered horizontally and vertically, appearing in white.

Introducing Conforma

Conforma

verify artifacts, enforce policies

- conforma.dev
- github.com/conforma
- [Sigstore](#) for verification
- [Open Policy Agent/Rego](#) for the policy engine



Why Conformal?

Goals:

- One tool to make it easier to close the gap
- Verify artifacts by default
- “Policy as code”
 - Bring DevOps and GitOps concepts to policy management
 - Not just the Rego policies, but also managing different configurations, and managing exceptions
- Human readable output
 - We want violations and warnings to be understandable and actionable, not “expected ‘foo’ to equal ‘bar’ on line 345” (for example)

Conforma today

At Red Hat:

- It's a key part of Red Hat's build/release CI
- Every container Red Hat builds must pass roughly 175 policy checks
- Originated as a component of  **KONFLUX**
 - See <https://konflux-ci.dev/>
- Now spun off into its own independent project

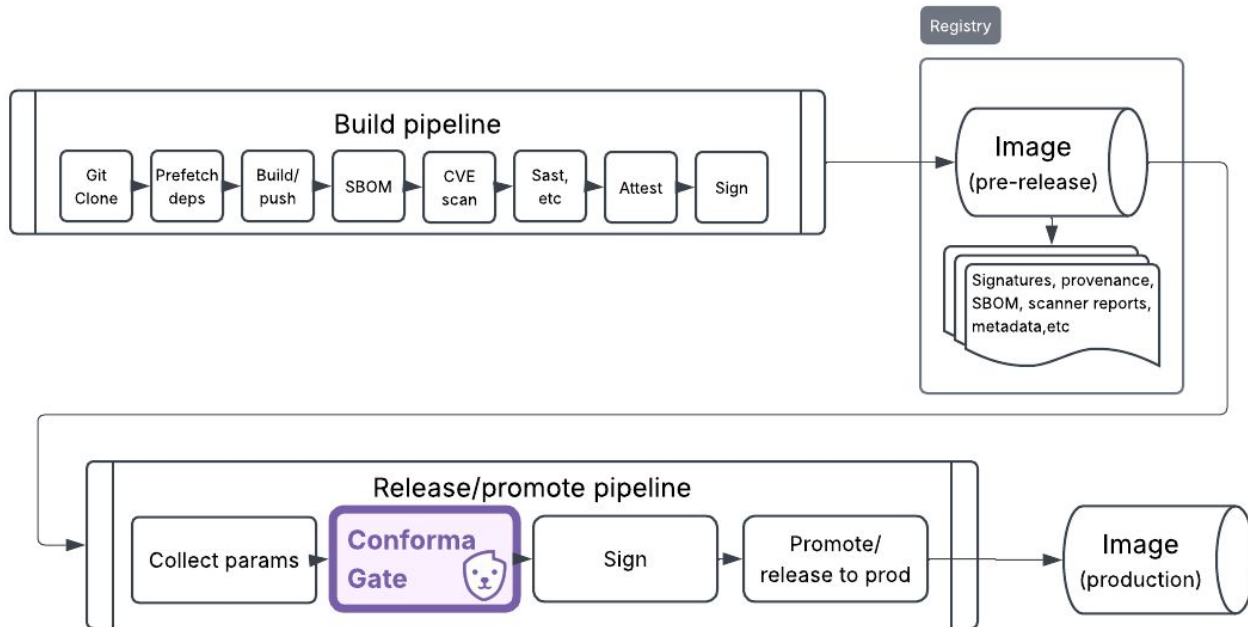
Conforma today

Outside Red Hat:

- Lots of potential...
- We think it pairs really well with Sigstore
- If you're creating SLSA provenance attestations, SBOMs, etc, Conforma is a great way to make use of that data and close the gap.
- Maybe you'll try it!

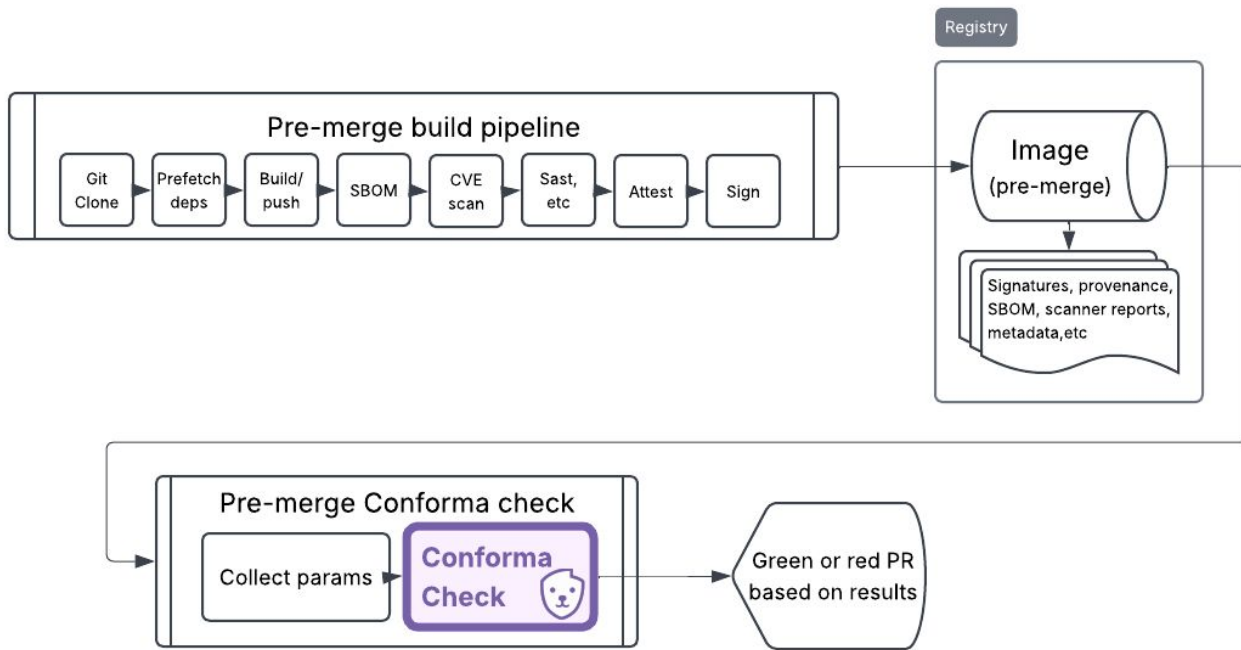
Pipeline examples

Gating a release/promotion



- The build pipeline creates the image and all the useful data we've talked about
- The "Conforma Gate" task looks at the image and the data and blocks the release if any problems are found
- Any problems produce human-readable and hopefully actionable output

Gating a merge, or “shift-left”



- Instead of building post-merge, this build pipeline is triggered by a PR
- The “Conforma Check” runs as a check in GitHub (for example)
- Any problems detected can block the PR from being merged, or at least indicate the build is not yet release-ready.

The background is a solid light purple color. Overlaid on this are several geometric shapes in different shades of purple. A large, dark purple circle is positioned on the left side. A medium-sized, medium-purple circle is in the center. A smaller, bright purple circle is on the right. A dark purple square is centered within the medium purple circle. The text "Conforma demo time" is written in white, bold, sans-serif font across the center of the image, overlapping the medium purple circle and the dark purple square.

Conforma demo time

Demo 1

Validate any data, not just container images

- **Any YAML or JSON.** The rule is not tied to a container schema.
- **Clear feedback.** The violation includes a human-readable message.
- **Adopt gradually.** Warnings can precede hard failures.



Demo 2

A valid signature does not make every claim trustworthy

- **Signed $\neq$ trusted.** Ownership is not the same as release compliance.
- **Correlation check.** Conforma compares declared Git coordinates with SLSA materials.
- **Hard failure.** A mismatch stops the artifact before promotion.

Demo 3

Tune an organization rule without rewriting Rego

- **Stable logic.** The Rego rule stays unchanged.
- **Dynamic configuration.** Teams update allowed values in policy data.
- **Visible decision.** The output explains exactly why the image failed.

Demo 4

Future-dated policies avoid flag-day surprises

- **Warn first.** Developers see the new requirement before it blocks.
- **Escalate automatically.** The same rule becomes a violation on its effective date.
- **Keep delivery moving.** No manual pipeline switch is needed on rollout day.





<https://conforma.dev>



<https://github.com/cuipinghuo/devconf-us-2026>

Thanks!

Cuiping Huo

Senior Software Quality Engineer

Red Hat

Simon Baird

Principal Software Engineer

Red Hat